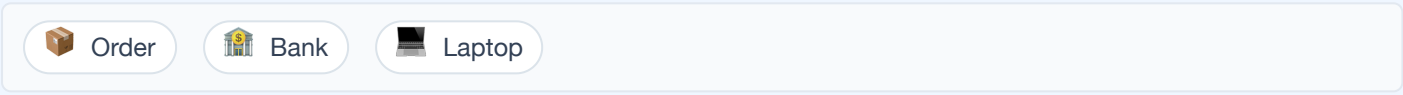
Optional Enrichment: Professional Python Design Choices

> **Not a standard teaching module.** Optional enrichment — use after core Advanced Python, Saturday labs, or interview prep only.
> Does **not** follow full Try these shape — optional enrichment, not on the critical path.

Previous: File Handling & Logging (finish core Advanced first)
Next: Continue calendar — Git & GitHub or MySQL phase as scheduled

Module Outcome:

- Function design — mutable default trap
- Composition before inheritance
- Abstract base classes (recap)
- `__str__` vs `__repr__`
- Custom exceptions with context



Where used?
- Order / notification flows — swap Email vs SMS without rewriting the service
- Inventory — reject oversell with a clear custom exception
- Payment demos — shared `pay()` contract (taught in OOP Part 4)
- Library APIs — `__str__` for humans, `__repr__` for debugging dumps

1. Function design: avoid the mutable default trap

This is more useful in real projects than memorising many argument varieties.

```
# Wrong: the same list is reused between calls.
def add_skill(skill, skills=[]):
    skills.append(skill)
    return skills

# Correct: create a fresh list when no list was supplied.
def add_skill(skill, skills=None):
    if skills is None:
        skills = []
    skills.append(skill)
    return skills

print(add_skill("Python"))  # ['Python']
print(add_skill("SQL"))     # ['SQL'], not ['Python', 'SQL']
```

Interview explanation: Default argument values are evaluated once when the function is defined. A mutable default can therefore leak state into later calls.

> **Remember:** Mutable defaults (`skills=[]`) are shared across calls — use `None` and create a fresh list inside.
> Call `add_skill` twice with a mutable default list `skills=[]` and print both results so the leak is visible.

```
def add_skill(skill, skills=[]):
    skills.append(skill)
    return skills

print(add_skill("Python"))
print(add_skill("SQL"))  # leaks prior call
```

Expected Output:

```
['Python']
['Python', 'SQL']
```

Fix `add_skill` by defaulting `skills=None` and creating a new list inside. Print two independent calls.

```
def add_skill(skill, skills=None):
    if skills is None:
        skills = []
    skills.append(skill)
    return skills

print(add_skill("Python"))
print(add_skill("SQL"))
```

Expected Output:

```
['Python']
['SQL']
```

2. Composition before inheritance

Use inheritance only when the child genuinely **is a** specialised version of the parent. Use composition when an object **has a** collaborator.

```
class EmailNotifier:
    def send(self, message):
        print(f"EMAIL: {message}")

class OrderService:
    def __init__(self, notifier):
        self.notifier = notifier  # OrderService HAS-A notifier

    def place_order(self, order_id):
        self.notifier.send(f"Order {order_id} placed")

service = OrderService(EmailNotifier())
service.place_order("ORD-101")
```

`OrderService` is not an `EmailNotifier`; it uses one. This design is easier to change later—for example, to SMS or WhatsApp notifications.

Common Error: Forcing inheritance when the relationship is only HAS-A — you end up with a rigid tree that is hard to swap (Email → SMS). Prefer composition.

Write `OrderService` that HAS-A `SmsNotifier` and call `place_order("ORD-202")`.

```
class SmsNotifier:
    def send(self, message):
        print(f"SMS: {message}")

class OrderService:
    def __init__(self, notifier):
        self.notifier = notifier

    def place_order(self, order_id):
        self.notifier.send(f"Order {order_id} placed")

OrderService(SmsNotifier()).place_order("ORD-202")
```

Expected Output:

```
SMS: Order ORD-202 placed
```

Start `OrderService` with `EmailNotifier`, place an order, then replace `svc.notifier` at runtime and place a second order.

```
class EmailNotifier:
    def send(self, message):
        print(f"EMAIL: {message}")

class OrderService:
    def __init__(self, notifier):
        self.notifier = notifier

    def place_order(self, order_id):
        self.notifier.send(f"Order {order_id} placed")

svc = OrderService(EmailNotifier())
svc.place_order("ORD-1")
svc.notifier = type("N", (), {"send": staticmethod(lambda m: print("LOG:", m))})()
svc.place_order("ORD-2")
```

3. Abstract base classes: recap (full notes in OOP Part 4)

ABC is taught in OOP Part 4 — Abstract Classes & Interfaces. This is a short Saturday recap only.

Use an abstract base class when multiple implementations must offer the same behaviour. This enrichment file is **after** Exceptions, so `raise` is allowed here.

```
from abc import ABC, abstractmethod

class PaymentGateway(ABC):
    @abstractmethod
    def pay(self, amount):
        """Return a payment result for the requested amount."""

class DemoGateway(PaymentGateway):
```

```
    def pay(self, amount):
        if amount <= 0:
            raise ValueError("Amount must be positive")
        return {"status": "success", "amount": amount}

print(DemoGateway().pay(499))
```

4. `__str__` vs `__repr__`

```
class Product:
    def __init__(self, sku, name, price):
        self.sku = sku
        self.name = name
        self.price = price

    def __str__(self):
        return f"{self.name} (₹{self.price})"

    def __repr__(self):
        return f"Product({self.sku!r}, {self.name!r}, {self.price!r})"

p = Product("P1", "Mouse", 799)
print(str(p))
print(repr(p))
```

Expected Output:

```
Mouse (₹799)
Product('P1', 'Mouse', 799)
```

5. Custom exceptions with context

```
class InsufficientStockError(Exception):
    def __init__(self, sku, requested, available):
        self.sku = sku
        self.requested = requested
        self.available = available
        super().__init__(
            f"{sku}: need {requested}, only {available} left"
        )

try:
    raise InsufficientStockError("P1", 5, 2)
except InsufficientStockError as e:
    print(e)
```

Mini-Project (Assignment): Stock reserve + notify

Business request: A shop needs to reserve stock and notify a customer after an order is accepted.

Must build

- `Product` with name, price, and stock.
- `reserve(quantity)` that rejects invalid or unavailable quantities.

- `InsufficientStockError` with a clear message.
- `ConsoleNotifier` used through composition by `OrderService`.
- `__str__` or `__repr__` useful during debugging.

Done when

- [ ] Reserve two units from stock five; remaining stock is three
- [ ] Reserve more than stock; clear message; stock unchanged
- [ ] Reserve zero or negative; rejected
- [ ] Run file directly; import without auto-placing an order

Stretch

- Add a `PaymentGateway` abstract base class and a `DemoGateway` implementation (see OOP Part 4 if this is new).
- Write two unit tests for `reserve` (kit 26_unittest_utils ideas).

How to run this session

Show a long `if/elif` program first, then ask what breaks when the business adds a second payment method or notification channel — then refactor toward composition.